

✓ classical_model_0

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5
6 from torchvision import datasets, transforms
7 from torch.utils.data import DataLoader
```

```
1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 print("Using device:", device)
```

Using device: cuda

```
1 transform = transforms.Compose([
2     transforms.ToTensor()
3 ])
4
5 train_dataset = datasets.MNIST(
6     root="./data",
7     train=True,
8     download=True,
9     transform=transform
10 )
11
12 test_dataset = datasets.MNIST(
13     root="./data",
14     train=False,
15     download=True,
16     transform=transform
17 )
18
19 train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
20 test_loader = DataLoader(test_dataset, batch_size=256, shuffle=False)
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 21.0MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 487kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 4.55MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 9.13MB/s]
```

```
1 train_dataset
```

```
Dataset MNIST
  Number of datapoints: 60000
  Root location: ./data
  Split: Train
  StandardTransform
Transform: Compose(
  ToTensor()
)
```

```
1 one_set = next(iter(train_loader))
2 one_set[0].shape
```

torch.Size([128, 1, 28, 28])

```
1 one_set = next(iter(test_loader))
2 one_set[0].shape
```

torch.Size([256, 1, 28, 28])

```
1 one_set[0].dtype
```

torch.float32

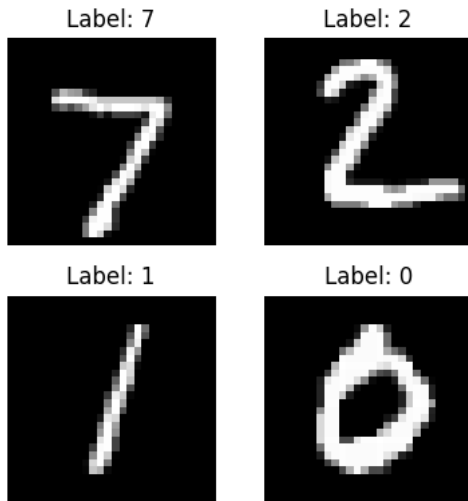
```
1 import matplotlib.pyplot as plt
```

```
1 images_batch, labels_batch = one_set
2
3 num_samples = 4
4 rows = 2
5 cols = 2
```

```

6
7 plt.figure(figsize=(cols * 2, rows * 2))
8
9 for i in range(num_samples):
10     image = images_batch[i]
11     label = labels_batch[i]
12
13     plt.subplot(rows, cols, i + 1)
14     plt.imshow(image.squeeze(), cmap="gray")
15     plt.title(f"Label: {label.item()}")
16     plt.axis("off")
17
18 plt.tight_layout()
19 plt.show()

```



```
1 image.shape
```

```
torch.Size([1, 28, 28])
```

```
1 images_batch.shape
```

```
torch.Size([256, 1, 28, 28])
```

```
1 images_batch.size(0)
```

```
256
```

```
1 images_batch.view(images_batch.size(0), -1).shape
```

```
torch.Size([256, 784])
```

```
1 images_batch.view(images_batch.size(0), -1).shape
```

```
torch.Size([256, 784])
```

```

1 class BSplineBasis(nn.Module):
2     def __init__(self, num_basis=10, xmin=0.0, xmax=1.0):
3         super().__init__()
4         self.num_basis = num_basis
5         self.register_buffer(
6             "knots",
7             torch.linspace(xmin, xmax, num_basis)
8         )
9         self.width = (xmax - xmin) / (num_basis - 1)
10
11     def forward(self, x):
12         # x: (B, in_dim)
13         x = x.unsqueeze(-1) # (B, in_dim, 1)
14         diff = torch.abs(x - self.knots)
15         basis = torch.clamp(1 - diff / self.width, min=0.0)
16         return basis # (B, in_dim, num_basis)

```

```

1 layer = BSplineBasis()
2 images_flat = images_batch.view(images_batch.size(0), -1)
3 single_image = images_flat[0].unsqueeze(0)

```

```
1 single_image.shape
```

```
torch.Size([1, 784])
```

```
1 single_image.squeeze(0).reshape(28, 28).unsqueeze(dim=-1).shape
```

```
torch.Size([28, 28, 1])
```

```
1 output = layer(single_image)
```

```
1 output
```

```
tensor([[[[1., 0., 0., ..., 0., 0., 0.],
          [1., 0., 0., ..., 0., 0., 0.],
          [1., 0., 0., ..., 0., 0., 0.],
          ...,
          [1., 0., 0., ..., 0., 0., 0.],
          [1., 0., 0., ..., 0., 0., 0.],
          [1., 0., 0., ..., 0., 0., 0.]]]])
```

```
1 output.shape
```

```
torch.Size([1, 784, 10])
```

Increasing number of basis (features per pixel, each pixel expanded into num_basis number of values):

- More expressive
- Can approximate more complex curves
- Higher resolution nonlinear modeling

but

- More memory
- More compute
- risk of overfitting
- Slower training

```
1 layer = BSplineBasis()
2 images_flat = images_batch.view(images_batch.size(0), -1)
3 output = layer(images_flat)
```

```
1 output.shape
```

```
torch.Size([256, 784, 10])
```

out = (basis_vals * coeff).sum(-1) is the edge function which calculates:

$$\phi_{ij}(x) = \sum_k c_{ijk} B_k(x)$$

model0 edge function:

$$\phi_{ij}(x) = \sum_k c_{ijk} B_k(x)$$

k = number of basis

B(x) = spline basis

c = learnable coefficient

```
1 class KANLayer(nn.Module):
2     def __init__(self, in_dim, out_dim, num_basis=10):
3         super().__init__()
4         self.in_dim = in_dim
5         self.out_dim = out_dim
6         self.num_basis = num_basis
7
8         self.basis = BSplineBasis(num_basis)
9
10        self.coeff = nn.Parameter(
11            torch.randn(in_dim, out_dim, num_basis) * 0.01
12        )
13
14        self.bias = nn.Parameter(torch.zeros(out_dim))
15        self.scale = nn.Parameter(torch.ones(out_dim))
16
```

```

17 def forward(self, x):
18     basis_vals = self.basis(x)
19     basis_vals = basis_vals.unsqueeze(2)
20     coeff = self.coeff.unsqueeze(0)
21
22     out = (basis_vals * coeff).sum(-1)
23     out = out.sum(1)
24
25     out = self.scale * out + self.bias
26
27     return out

```

```

1 kan_layer_instance = KANLayer(in_dim=784, out_dim=64, num_basis=10).to(device)
2 single_image_on_device = single_image.to(device)
3 kan_output = kan_layer_instance(single_image_on_device)

```

```

1 print("Output from KANLayer:")
2 print(kan_output)
3 print("\nShape of KANLayer output:")
4 print(kan_output.shape)

```

Output from KANLayer:

```

tensor([[ 0.1448, -0.3409,  0.2044, -0.2480, -0.0696,  0.0897,  0.3178, -0.0823,
         -0.1812, -0.1344, -0.3737,  0.1547,  0.0742,  0.0826,  0.2787,  0.2859,
         -0.2218, -0.3199,  0.1112,  0.0099,  0.1035, -0.2354, -0.2264, -0.1509,
          0.0396, -0.2184,  0.3747, -0.0251, -0.1113,  0.1657,  0.7489,  0.0922,
          0.0215,  0.0358, -0.1590,  0.4388, -0.1337, -0.3266,  0.1774,  0.1693,
          0.1531,  0.1121, -0.2756, -0.2985,  0.2371,  0.0778,  0.3425, -0.1021,
         -0.4581, -0.0178, -0.3215, -0.1606,  0.1051, -0.1017,  0.0068, -0.1017,
          0.1976,  0.0875, -0.0557, -0.4838, -0.1097,  0.2616, -0.1948,  0.2235]],
        device='cuda:0', grad_fn=<AddBackward0>)

```

Shape of KANLayer output:
torch.Size([1, 64])

```

1 class KANModel(nn.Module):
2     def __init__(self):
3         super().__init__()
4
5         self.kan1 = KANLayer(784, 64, num_basis=8)
6         self.norm1 = nn.LayerNorm(64)
7
8         self.kan2 = KANLayer(64, 32, num_basis=8)
9         self.norm2 = nn.LayerNorm(32)
10
11        self.fc = nn.Linear(32, 10)
12
13    def forward(self, x):
14        x = x.view(x.size(0), -1)
15
16        x = self.kan1(x)
17        x = self.norm1(x)
18        x = F.relu(x)
19
20        x = self.kan2(x)
21        x = self.norm2(x)
22        x = F.relu(x)
23
24        x = self.fc(x)
25        return x

```

```

1 model_instance = KANModel().to(device)
2 single_image_on_device = single_image.to(device)
3 model_output = model_instance(single_image_on_device)
4
5 print("Output from KANModel:")
6 print(model_output)
7 print("\nShape of KANModel output:")
8 print(model_output.shape)

```

Output from KANModel:

```

tensor([[ -0.5265, -0.5821,  0.6258,  0.1865, -0.1033,  0.0048, -0.6104,  0.8780,
          -0.3063, -0.3472]], device='cuda:0', grad_fn=<AddmmBackward0>)

```

Shape of KANModel output:
torch.Size([1, 10])

```
1 model_classical_0 = KANModel().to(device)
2
3 criterion = nn.CrossEntropyLoss()
4 optimizer = optim.AdamW(model_classical_0.parameters(), lr=3e-4)
```

```
1 print(model_classical_0)
```

```
KANModel(
  (kan1): KANLayer(
    (basis): BSplineBasis()
  )
  (norm1): LayerNorm((64,), eps=1e-05, elementwise_affine=True)
  (kan2): KANLayer(
    (basis): BSplineBasis()
  )
  (norm2): LayerNorm((32,), eps=1e-05, elementwise_affine=True)
  (fc): Linear(in_features=32, out_features=10, bias=True)
)
```

```
1 total_params = sum(p.numel() for p in model_classical_0.parameters() if p.requires_grad)
2 print(f"Total trainable parameters in model_classical_0: {total_params:,}")
```

```
Total trainable parameters in model_classical_0: 418,506
```

```
1 def train_epoch(model, loader):
2     model.train()
3     total_loss = 0
4     correct = 0
5     total = 0
6
7     for images, labels in loader:
8         images = images.to(device)
9         labels = labels.to(device)
10
11     optimizer.zero_grad()
12
13     outputs = model(images)
14     loss = criterion(outputs, labels)
15
16     loss.backward()
17     optimizer.step()
18
19     total_loss += loss.item()
20
21     _, predicted = outputs.max(1)
22     total += labels.size(0)
23     correct += predicted.eq(labels).sum().item()
24
25     return total_loss / len(loader), 100. * correct / total
```

```
1 def evaluate(model, loader):
2     model.eval()
3
4     total_loss = 0
5     correct = 0
6     total = 0
7
8     with torch.no_grad():
9         for images, labels in loader:
10             images = images.to(device)
11             labels = labels.to(device)
12
13             outputs = model(images)
14             loss = criterion(outputs, labels)
15
16             total_loss += loss.item()
17
18             _, predicted = outputs.max(1)
19             total += labels.size(0)
20             correct += predicted.eq(labels).sum().item()
21
22     avg_loss = total_loss / len(loader)
23     accuracy = 100. * correct / total
24
25     return avg_loss, accuracy
```

```

1 epochs = 15
2 best_test_acc = 0.0
3
4 res0 = {
5     "train_loss": [],
6     "train_acc": [],
7     "test_acc": [],
8     "test_loss": []
9 }
10
11 for epoch in range(epochs):
12     train_loss, train_acc = train_epoch(model_classical_0, train_loader)
13     test_loss, test_acc = evaluate(model_classical_0, test_loader)
14
15     res0["train_loss"].append(train_loss)
16     res0["train_acc"].append(train_acc)
17     res0["test_loss"].append(test_loss)
18     res0["test_acc"].append(test_acc)
19
20     print(f"Epoch {epoch+1:02d} | "
21           f"Loss: {train_loss:.4f} | "
22           f"Train Acc: {train_acc:.2f}% | "
23           f"Test Acc: {test_acc:.2f}%")
24
25     if test_acc > best_test_acc:
26         best_test_acc = test_acc
27         torch.save(model_classical_0.state_dict(), 'best_model_classical_0.pth')
28     print(f" --> Model saved with new best test accuracy: {best_test_acc:.2f}%")

```

```

Epoch 01 | Loss: 0.8500 | Train Acc: 82.49% | Test Acc: 93.48%
--> Model saved with new best test accuracy: 93.48%
Epoch 02 | Loss: 0.2442 | Train Acc: 94.57% | Test Acc: 93.89%
--> Model saved with new best test accuracy: 93.89%
Epoch 03 | Loss: 0.1450 | Train Acc: 96.53% | Test Acc: 94.85%
--> Model saved with new best test accuracy: 94.85%
Epoch 04 | Loss: 0.1014 | Train Acc: 97.45% | Test Acc: 95.03%
--> Model saved with new best test accuracy: 95.03%
Epoch 05 | Loss: 0.0701 | Train Acc: 98.30% | Test Acc: 94.85%
Epoch 06 | Loss: 0.0559 | Train Acc: 98.60% | Test Acc: 95.19%
--> Model saved with new best test accuracy: 95.19%
Epoch 07 | Loss: 0.0475 | Train Acc: 98.77% | Test Acc: 95.33%
--> Model saved with new best test accuracy: 95.33%
Epoch 08 | Loss: 0.0328 | Train Acc: 99.13% | Test Acc: 95.12%
Epoch 09 | Loss: 0.0348 | Train Acc: 99.04% | Test Acc: 94.43%
Epoch 10 | Loss: 0.0329 | Train Acc: 99.05% | Test Acc: 95.73%
--> Model saved with new best test accuracy: 95.73%
Epoch 11 | Loss: 0.0167 | Train Acc: 99.60% | Test Acc: 95.57%
Epoch 12 | Loss: 0.0186 | Train Acc: 99.53% | Test Acc: 95.18%
Epoch 13 | Loss: 0.0176 | Train Acc: 99.53% | Test Acc: 95.33%
Epoch 14 | Loss: 0.0162 | Train Acc: 99.57% | Test Acc: 95.46%
Epoch 15 | Loss: 0.0160 | Train Acc: 99.55% | Test Acc: 94.99%

```

```

1 import matplotlib.pyplot as plt
2 import random
3 import math
4 import torch

```

```

1 def visualize_predictions(model, dataset, device, num_samples=16):
2     model.eval()
3
4     indices = random.sample(range(len(dataset)), num_samples)
5
6     cols = int(math.sqrt(num_samples))
7     rows = math.ceil(num_samples / cols)
8
9     plt.figure(figsize=(cols * 2, rows * 2))
10
11     for i, idx in enumerate(indices):
12         image, label = dataset[idx]
13         input_img = image.unsqueeze(0).to(device)
14
15         with torch.no_grad():
16             output = model(input_img)
17             pred = output.argmax(dim=1).item()
18
19         plt.subplot(rows, cols, i + 1)
20         plt.imshow(image.squeeze(), cmap="gray")
21
22         color = "green" if pred == label else "red"

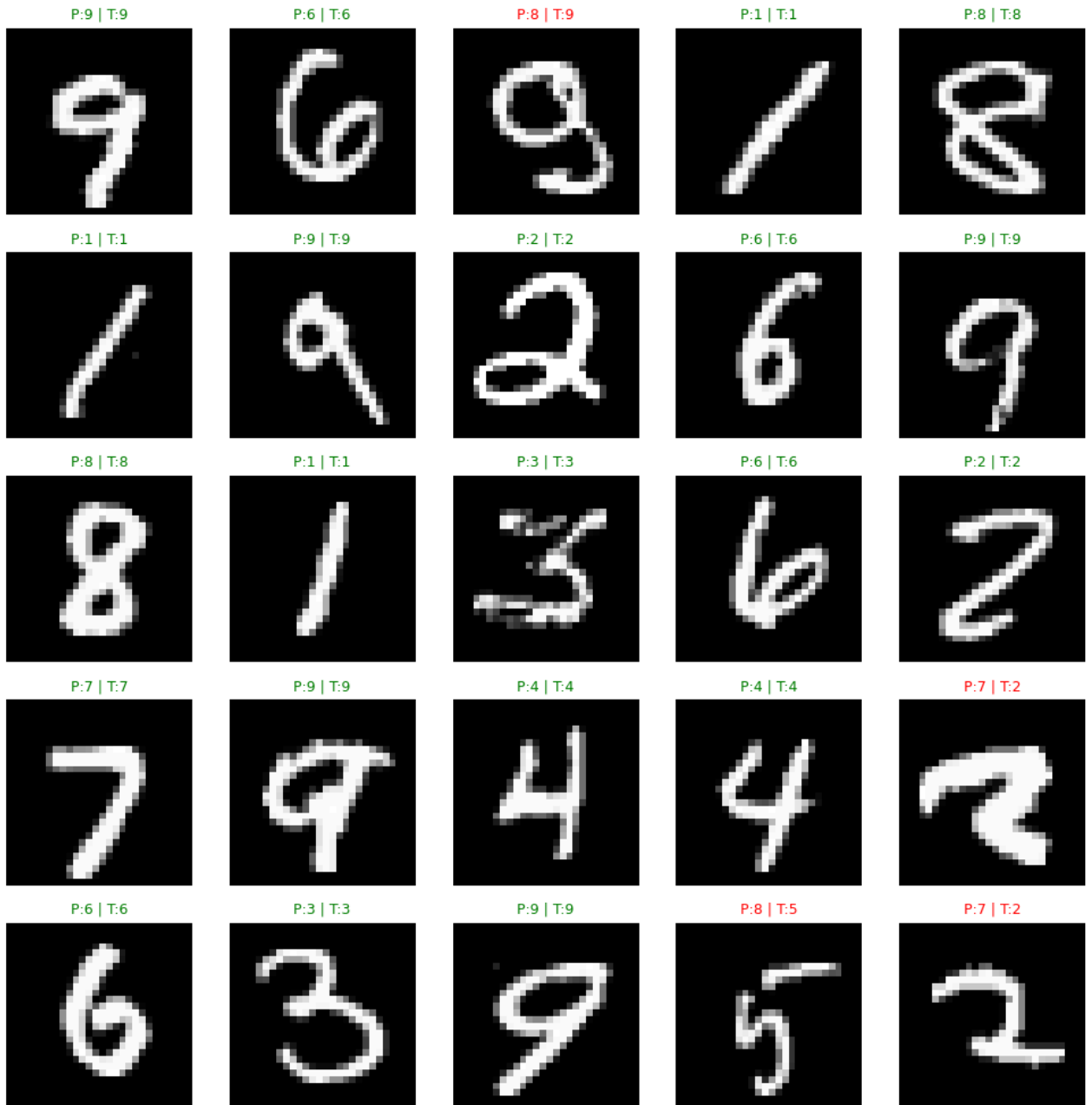
```

```

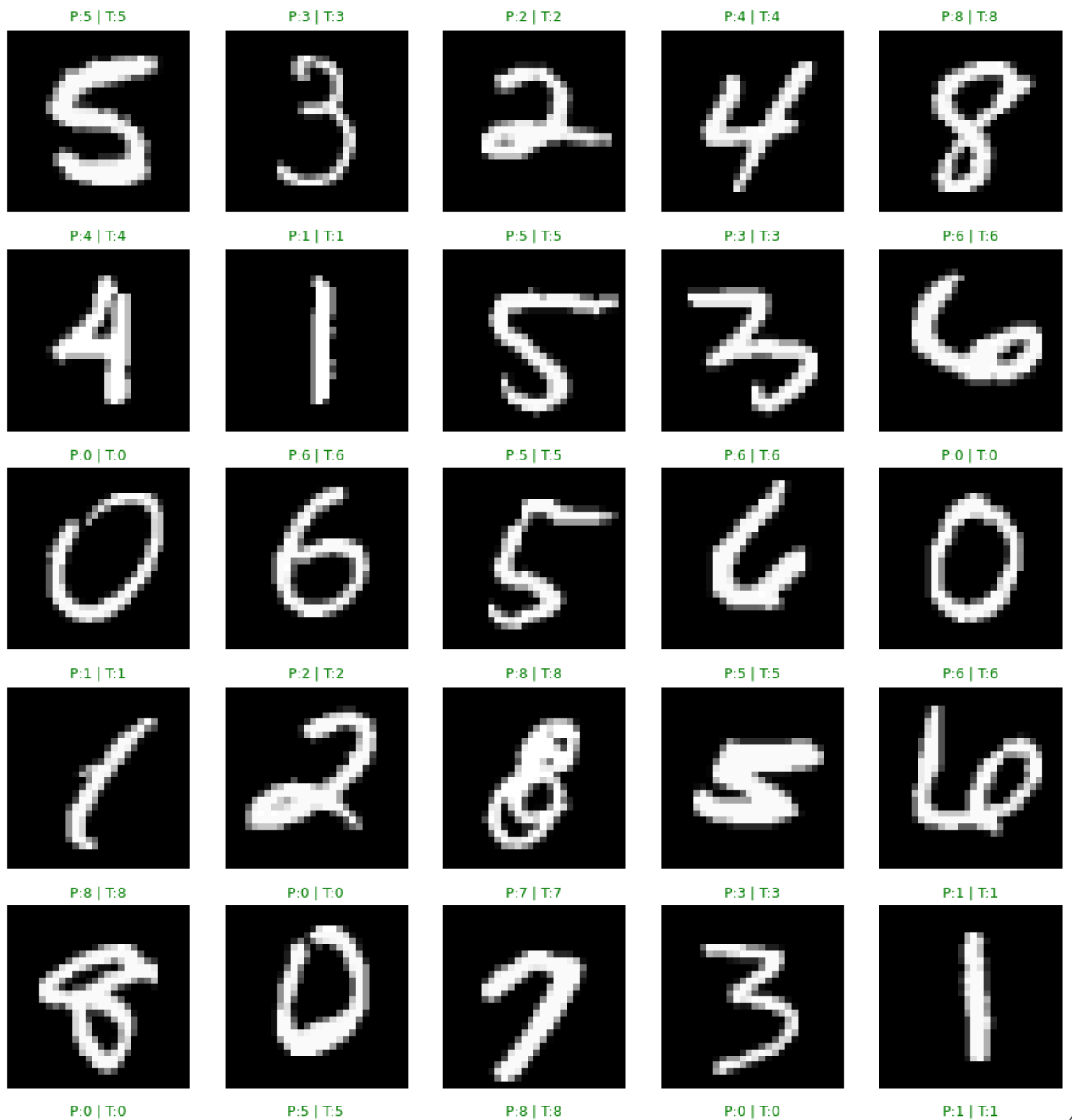
23 plt.title(f"P:{pred} | T:{label}", color=color, fontsize=9)
24 plt.axis("off")
25
26 plt.tight_layout()
27 plt.show()

```

```
1 visualize_predictions(model_classical_0, test_dataset, device, num_samples=25)
```



```
1 visualize_predictions(model_classical_0, test_dataset, device, num_samples=30)
```



```

1 def final_evaluation(model, loader, device):
2     model.eval()
3
4     criterion = nn.CrossEntropyLoss()
5
6     total_loss = 0
7     correct = 0
8     total = 0
9
10    num_classes = 10
11    confusion = torch.zeros(num_classes, num_classes)
12
13    with torch.no_grad():
14        for images, labels in loader:
15            images = images.to(device)
16            labels = labels.to(device)
17
18            outputs = model(images)
19            loss = criterion(outputs, labels)
20
21            total_loss += loss.item()
22
23            preds = outputs.argmax(dim=1)
24
25            total += labels.size(0)
26            correct += (preds == labels).sum().item()
27

```



```

28     for t, p in zip(labels.view(-1), preds.view(-1)):
29         confusion[t.long(), p.long()] += 1
30
31     avg_loss = total_loss / len(loader)
32     accuracy = 100. * correct / total
33
34     print(f"\nFinal Test Loss: {avg_loss:.4f}")
35     print(f"Final Test Accuracy: {accuracy:.2f}%\n")
36
37     print("Per-class accuracy:")
38     for i in range(num_classes):
39         class_acc = 100. * confusion[i, i] / confusion[i].sum()
40         print(f"Class {i}: {class_acc:.2f}%")
41
42     plt.figure(figsize=(8,6))
43     plt.imshow(confusion.numpy(), interpolation='nearest')
44     plt.title("Confusion Matrix")
45     plt.colorbar()
46     plt.xlabel("Predicted")
47     plt.ylabel("True")
48     plt.xticks(range(num_classes))
49     plt.yticks(range(num_classes))
50     plt.show()
51
52     return accuracy

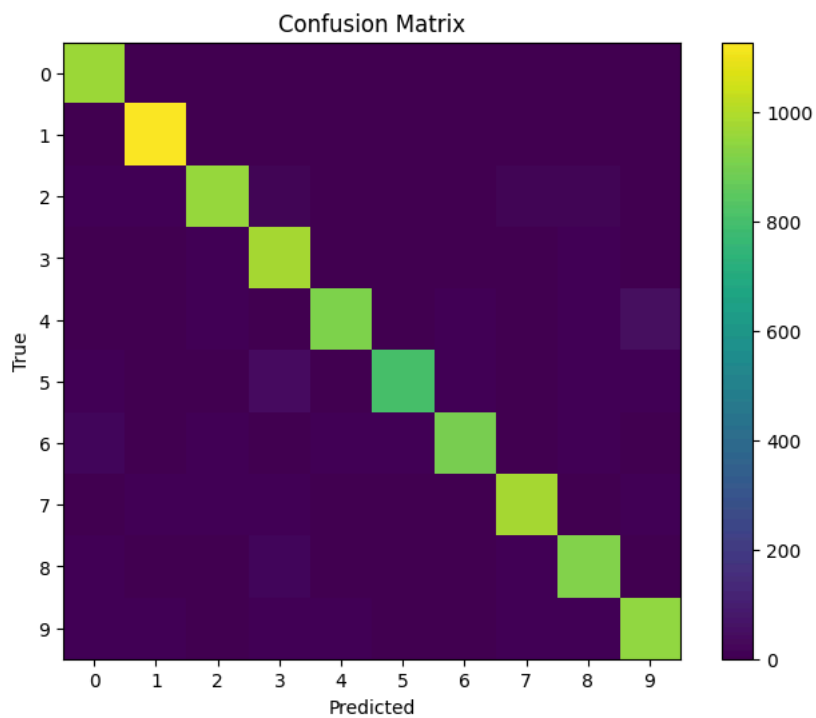
```

```
1 final_evaluation(model_classical_0, test_loader, device)
```

Final Test Loss: 0.1950
Final Test Accuracy: 94.99%

Per-class accuracy:

Class 0: 98.78%
Class 1: 99.38%
Class 2: 92.93%
Class 3: 97.03%
Class 4: 92.67%
Class 5: 90.36%
Class 6: 94.15%
Class 7: 95.43%
Class 8: 94.25%
Class 9: 93.86%



94.99

classical_model_1

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim

```

```

4 import torch.nn.functional as F
5 import math
6
7 from torchvision import datasets, transforms
8 from torch.utils.data import DataLoader
9
10 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
11 print("Using device:", device)

```

Using device: cuda

```

1 transform = transforms.Compose([
2     transforms.ToTensor(),
3     transforms.Normalize((0.1307,), (0.3081,))
4 ])
5
6 train_dataset = datasets.MNIST(
7     root="./data",
8     train=True,
9     download=True,
10    transform=transform
11 )
12
13 test_dataset = datasets.MNIST(
14     root="./data",
15     train=False,
16     download=True,
17     transform=transform
18 )

```

```

1 train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
2 test_loader = DataLoader(test_dataset, batch_size=256, shuffle=False)

```

model1 edge function:

$$y = w_b \text{Linear}(\text{SiLU}(x)) + w_s \text{Spline}(x)$$

w_b = base scale

w_s = spline scale

$\text{spline}(x) = \text{torch.einsum}('bik,iok \rightarrow bo', \text{basis_vals}, \text{self.coeff})$ (weighted sum of spline basis functions)

```

1 class KANLayerV1(nn.Module):
2     def __init__(self, in_dim, out_dim, num_basis=10, xmin=-3.0, xmax=3.0):
3         super().__init__()
4
5         self.in_dim = in_dim
6         self.out_dim = out_dim
7         self.num_basis = num_basis
8
9         self.register_buffer("knots", torch.linspace(xmin, xmax, num_basis))
10        self.width = (xmax - xmin) / (num_basis - 1)
11
12        std = 1.0 / math.sqrt(in_dim)
13
14        self.coeff = nn.Parameter(
15            torch.empty(in_dim, out_dim, num_basis).uniform_(-std, std)
16        )
17
18        self.base_weight = nn.Parameter(
19            torch.empty(out_dim, in_dim).uniform_(-std, std)
20        )
21        self.base_bias = nn.Parameter(
22            torch.empty(out_dim).uniform_(-std, std)
23        )
24
25        self.base_activation = nn.SiLU()
26
27        self.base_scale = nn.Parameter(torch.ones(out_dim))
28        self.spline_scale = nn.Parameter(torch.ones(out_dim))
29
30    def forward(self, x):
31        base = F.linear(self.base_activation(x), self.base_weight, self.base_bias)
32        base = self.base_scale * base
33
34        x_expanded = x.unsqueeze(-1)
35        diff = torch.abs(x_expanded - self.knots)

```

```

36     basis_vals = torch.clamp(1 - diff / self.width, min=0.0)
37
38     spline = torch.einsum('bik,iok->bo', basis_vals, self.coeff)
39     spline = self.spline_scale * spline
40
41     return base + spline

```

```

1  class KANModelV1(nn.Module):
2      def __init__(self):
3          super().__init__()
4          self.kan1 = KANLayerV1(784, 128, num_basis=10)
5          self.ln1 = nn.LayerNorm(128)
6
7          self.kan2 = KANLayerV1(128, 64, num_basis=10)
8          self.ln2 = nn.LayerNorm(64)
9
10         self.kan3 = KANLayerV1(64, 10, num_basis=10)
11
12     def forward(self, x):
13         x = x.view(x.size(0), -1)
14         x = self.kan1(x)
15         x = self.ln1(x)
16         x = self.kan2(x)
17         x = self.ln2(x)
18         x = self.kan3(x)
19         return x

```

```

1 model_classical_1 = KANModelV1().to(device)
2
3 criterion = nn.CrossEntropyLoss()
4 optimizer = optim.AdamW(model_classical_1.parameters(), lr=1e-3)

```

```

1 total_params = sum(p.numel() for p in model_classical_1.parameters() if p.requires_grad)
2 print(f"Total trainable parameters in model_classical_1: {total_params:,}")

```

Total trainable parameters in model_classical_1: 1,202,014

```

1 epochs = 15
2 best_test_acc = 0.0
3
4 res1 = {
5     "train_loss": [],
6     "train_acc": [],
7     "test_acc": [],
8     "test_loss": []
9 }
10
11 for epoch in range(epochs):
12     train_loss, train_acc = train_epoch(model_classical_1, train_loader)
13     test_loss, test_acc = evaluate(model_classical_1, test_loader)
14
15     res1["train_loss"].append(train_loss)
16     res1["train_acc"].append(train_acc)
17     res1["test_acc"].append(test_acc)
18     res1["test_loss"].append(test_loss)
19
20     print(f"Epoch {epoch+1:02d} | "
21           f"Loss: {train_loss:.4f} | "
22           f"Train Acc: {train_acc:.2f}% | "
23           f"Test Acc: {test_acc:.2f}%")
24
25     if test_acc > best_test_acc:
26         best_test_acc = test_acc
27         torch.save(model_classical_1.state_dict(), 'best_model_classical_1.pth')
28         print(f" --> Model saved with new best test accuracy: {best_test_acc:.2f}%")

```

```

Epoch 01 | Loss: 0.3441 | Train Acc: 90.63% | Test Acc: 95.35%
--> Model saved with new best test accuracy: 95.35%
Epoch 02 | Loss: 0.1126 | Train Acc: 96.61% | Test Acc: 96.68%
--> Model saved with new best test accuracy: 96.68%
Epoch 03 | Loss: 0.0729 | Train Acc: 97.68% | Test Acc: 96.62%
Epoch 04 | Loss: 0.0504 | Train Acc: 98.43% | Test Acc: 97.17%
--> Model saved with new best test accuracy: 97.17%
Epoch 05 | Loss: 0.0380 | Train Acc: 98.72% | Test Acc: 96.85%
Epoch 06 | Loss: 0.0290 | Train Acc: 99.07% | Test Acc: 97.26%

```

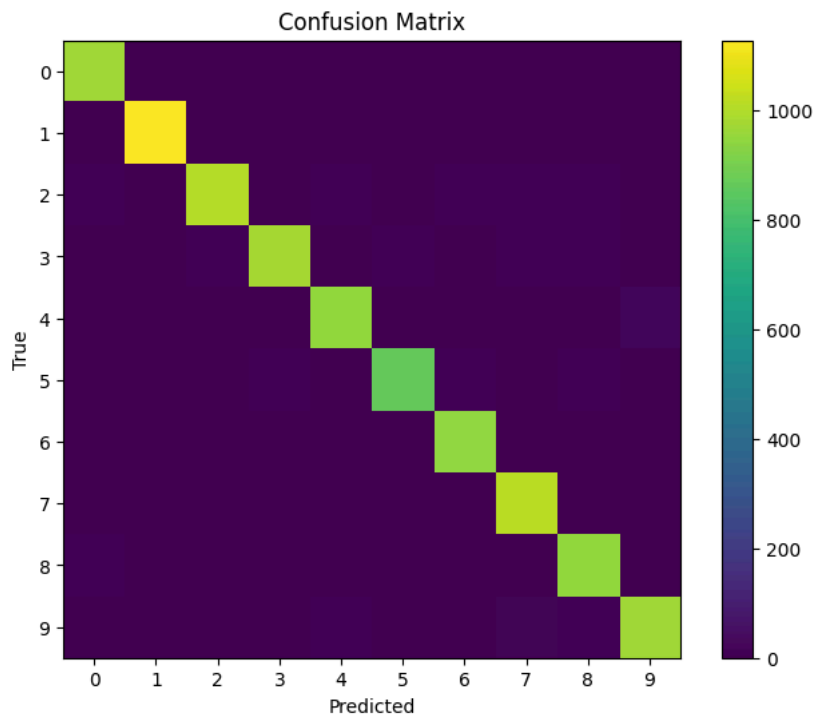
```
--> Model saved with new best test accuracy: 97.26%
Epoch 07 | Loss: 0.0255 | Train Acc: 99.17% | Test Acc: 97.75%
--> Model saved with new best test accuracy: 97.75%
Epoch 08 | Loss: 0.0208 | Train Acc: 99.33% | Test Acc: 97.35%
Epoch 09 | Loss: 0.0220 | Train Acc: 99.25% | Test Acc: 97.64%
Epoch 10 | Loss: 0.0182 | Train Acc: 99.37% | Test Acc: 97.87%
--> Model saved with new best test accuracy: 97.87%
Epoch 11 | Loss: 0.0162 | Train Acc: 99.45% | Test Acc: 97.80%
Epoch 12 | Loss: 0.0174 | Train Acc: 99.39% | Test Acc: 97.71%
Epoch 13 | Loss: 0.0128 | Train Acc: 99.55% | Test Acc: 98.22%
--> Model saved with new best test accuracy: 98.22%
Epoch 14 | Loss: 0.0106 | Train Acc: 99.64% | Test Acc: 97.76%
Epoch 15 | Loss: 0.0131 | Train Acc: 99.54% | Test Acc: 97.56%
```

```
1 final_evaluation(model_classical_1, test_loader, device)
```

```
Final Test Loss: 0.0928
Final Test Accuracy: 97.56%
```

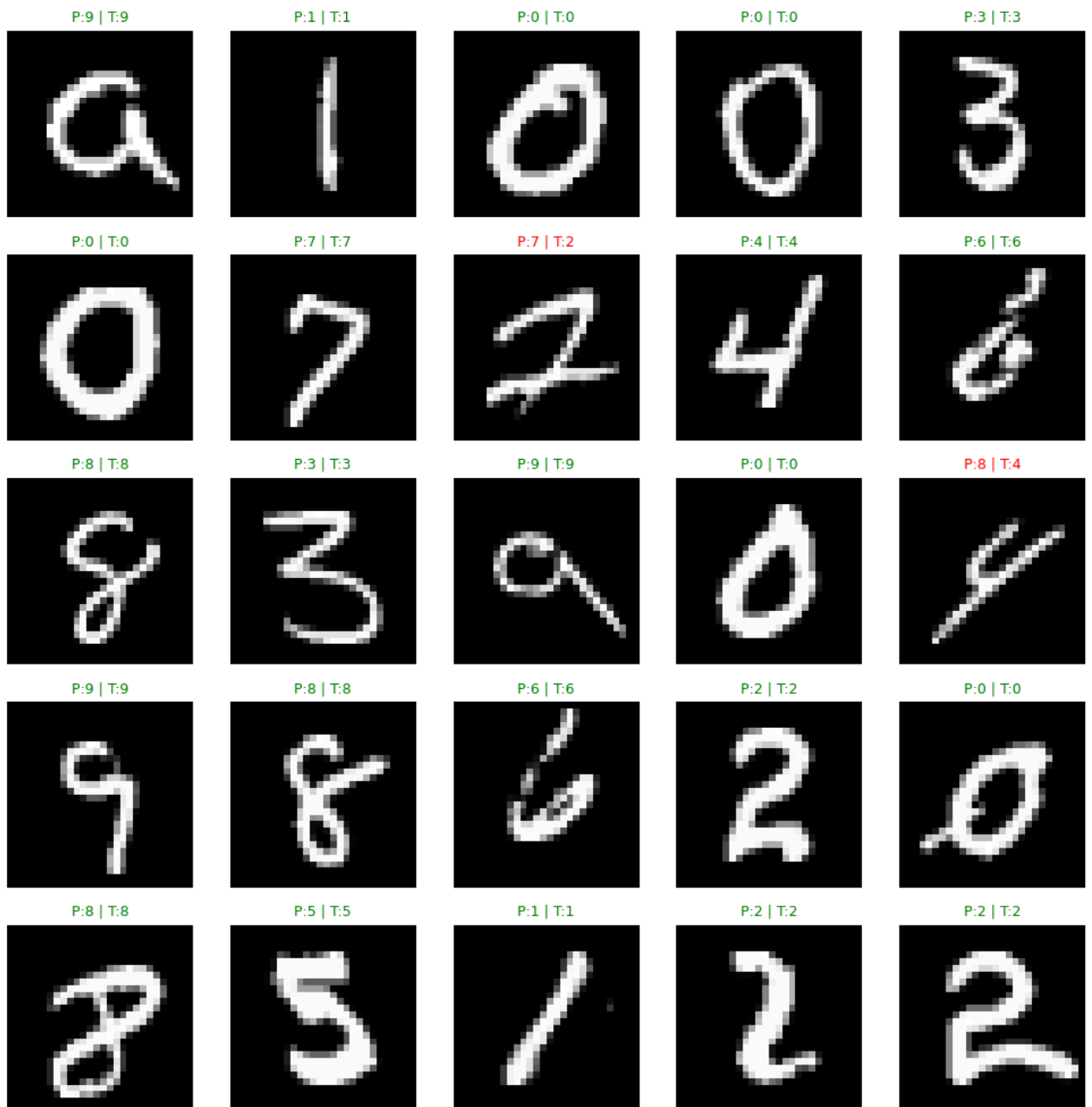
```
Per-class accuracy:
```

```
Class 0: 98.98%
Class 1: 99.30%
Class 2: 96.90%
Class 3: 96.63%
Class 4: 96.64%
Class 5: 96.30%
Class 6: 98.64%
Class 7: 98.44%
Class 8: 97.23%
Class 9: 96.23%
```



97.56

```
1 visualize_predictions(model_classical_1, test_dataset, device, num_samples=25)
```



comparison

```

1 import matplotlib.pyplot as plt
2
3 def plot_comparison(res0, res1,
4                     label0="Model 0",
5                     label1="Model 1"):
6
7     epochs = range(1, len(res0["test_loss"]) + 1)
8
9     fig, axes = plt.subplots(2, 1, figsize=(8, 10))
10
11     axes[0].plot(epochs, res0["test_loss"], label=f"{label0} Test Loss")
12     axes[0].plot(epochs, res1["test_loss"], label=f"{label1} Test Loss")
13     axes[0].set_title("Test Loss Comparison")
14     axes[0].set_xlabel("Epoch")
15     axes[0].set_ylabel("Loss")
16     axes[0].legend()
17     axes[0].grid(True)
18
19     axes[1].plot(epochs, res0["test_acc"], label=f"{label0} Test Accuracy")
20     axes[1].plot(epochs, res1["test_acc"], label=f"{label1} Test Accuracy")
21     axes[1].set_title("Test Accuracy Comparison")
22     axes[1].set_xlabel("Epoch")
23     axes[1].set_ylabel("Accuracy (%)")

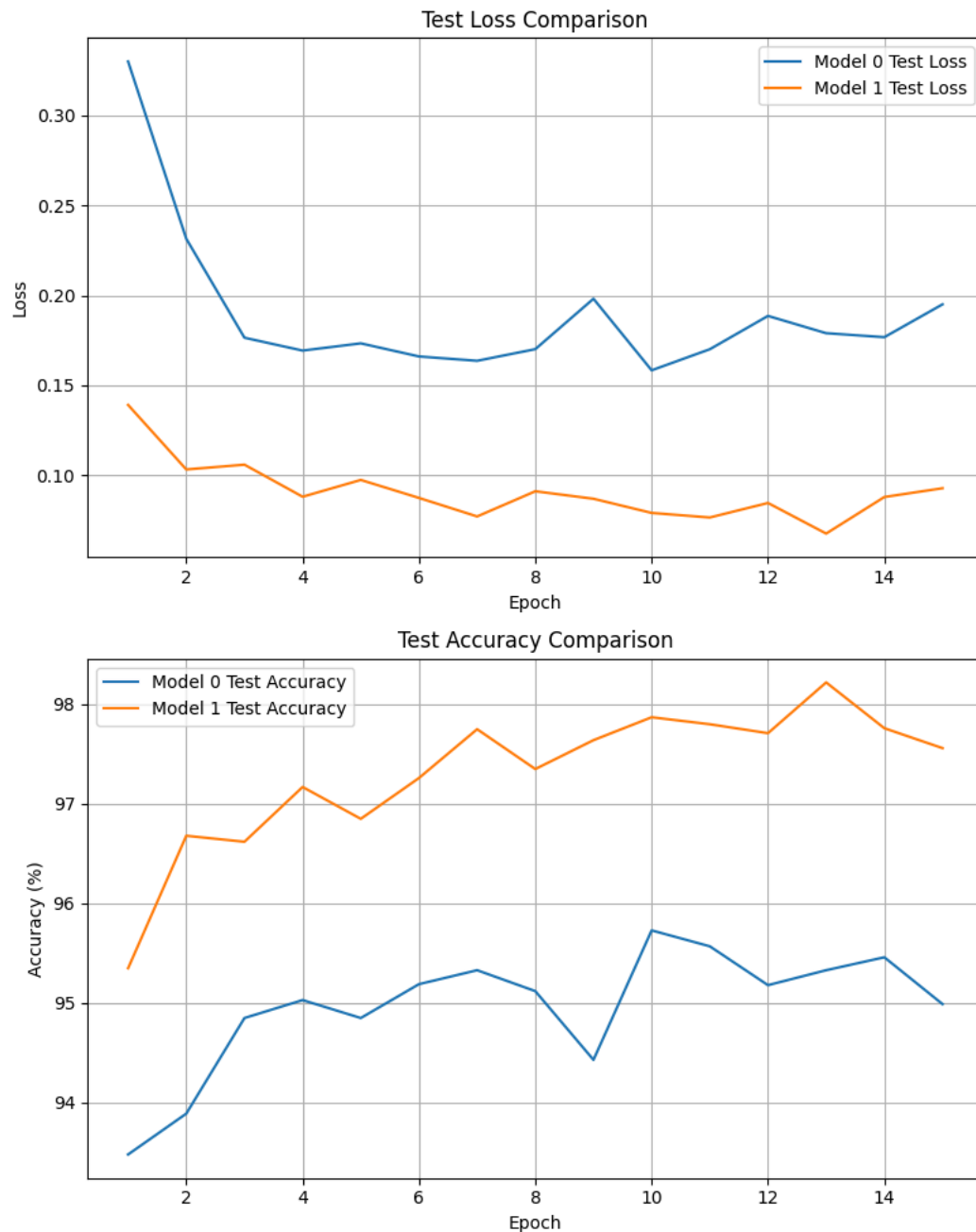
```

```

24 axes[1].legend()
25 axes[1].grid(True)
26
27 plt.tight_layout()
28 plt.show()

```

```
1 plot_comparison(res0, res1)
```



```
1 !pip install thop -q
```

```

1 import torch
2 import time
3 import pandas as pd
4 from thop import profile
5
6 def count_parameters(model):
7     return sum(p.numel() for p in model.parameters() if p.requires_grad)
8
9 def compute_macs(model, input_size=(1,1,28,28)):
10     dummy = torch.randn(input_size).to(device)
11     macs, _ = profile(model, inputs=(dummy,), verbose=False)
12     return macs
13
14 def measure_inference_time(model, input_size=(1,1,28,28), runs=200):
15     model.eval()
16     dummy = torch.randn(input_size).to(device)

```

```

17
18 for _ in range(20):
19     _ = model(dummy)
20
21 if device.type == "cuda":
22     torch.cuda.synchronize()
23
24 start = time.time()
25
26 for _ in range(runs):
27     _ = model(dummy)
28
29 if device.type == "cuda":
30     torch.cuda.synchronize()
31
32 end = time.time()
33
34 return (end - start) / runs
35
36 params0 = count_parameters(model_classical_0)
37 params1 = count_parameters(model_classical_1)
38
39 macs0 = compute_macs(model_classical_0)
40 macs1 = compute_macs(model_classical_1)
41
42 inf0 = measure_inference_time(model_classical_0)
43 inf1 = measure_inference_time(model_classical_1)
44
45 acc0 = res0["test_acc"][-1]
46 acc1 = res1["test_acc"][-1]
47
48 comparison_df = pd.DataFrame({
49     "Metric": [
50         "Parameters (M)",
51         "MACs (M)",
52         "Inference Time (ms)",
53         "Final Test Accuracy (%)"
54     ],
55     "KAN v0": [
56         params0 / 1e6,
57         macs0 / 1e6,
58         inf0 * 1000,
59         acc0
60     ],
61     "KAN v1": [
62         params1 / 1e6,
63         macs1 / 1e6,
64         inf1 * 1000,
65         acc1
66     ]
67 })

```

1 comparison_df

	Metric	KAN v0	KAN v1
0	Parameters (M)	0.418506	1.202014
1	MACs (M)	0.000704	0.000768
2	Inference Time (ms)	1.027068	1.518185
3	Final Test Accuracy (%)	94.990000	97.560000

classical_model_2

model2 edge function: base linear + Chebyshev polynomial

$$y = \sum W \sigma(x) + b + \sum_d c_d T_d(\tanh(x))$$

T_d = Chebyshev polynomial of degree d

c_d = trainable coefficient for each polynomial basis

tanh(x) = input normalization step

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import math
6 import time
7 import pandas as pd
8 from thop import profile
9
10 def count_parameters(model):
11     return sum(p.numel() for p in model.parameters() if p.requires_grad)
12
13 def compute_macs(model, input_size=(1,1,28,28)):
14     dummy = torch.randn(input_size).to(device)
15     macs, _ = profile(model, inputs=(dummy,), verbose=False)
16     return macs
17
18 def measure_inference_time(model, input_size=(1,1,28,28), runs=200):
19     model.eval()
20     dummy = torch.randn(input_size).to(device)
21
22     for _ in range(20):
23         _ = model(dummy)
24
25     if device == "cuda":
26         torch.cuda.synchronize()
27
28     start = time.time()
29
30     for _ in range(runs):
31         _ = model(dummy)
32
33     if device == "cuda":
34         torch.cuda.synchronize()
35
36     end = time.time()
37
38     return (end - start) / runs
39
40
41 class ChebyKANLayer(nn.Module):
42     def __init__(self, in_features, out_features, degree=3):
43         super().__init__()
44         self.in_features = in_features
45         self.out_features = out_features
46         self.degree = degree
47
48         self.base_activation = nn.SiLU()
49         self.base_weight = nn.Parameter(torch.Tensor(out_features, in_features))
50         self.base_bias = nn.Parameter(torch.Tensor(out_features))
51
52
53         self.poly_weight = nn.Parameter(torch.Tensor(out_features, in_features * (degree + 1)))
54
55         self.reset_parameters()
56
57     def reset_parameters(self):
58         nn.init.kaiming_uniform_(self.base_weight, a=math.sqrt(5))
59         nn.init.zeros_(self.base_bias)
60         nn.init.xavier_normal_(self.poly_weight)
61
62     def forward(self, x):
63         base_out = F.linear(self.base_activation(x), self.base_weight, self.base_bias)
64
65         x_norm = torch.tanh(x)
66
67         basis = [torch.ones_like(x_norm), x_norm]
68         for i in range(2, self.degree + 1):
69             basis.append(2 * x_norm * basis[-1] - basis[-2])
70
71         basis_tensor = torch.cat(basis, dim=1)
72
73         poly_out = F.linear(basis_tensor, self.poly_weight)
74
75         return base_out + poly_out
76
77 class KANModelV2(nn.Module):
78     def __init__(self):

```



```

79     super().__init__()
80     self.kan1 = ChebyKANLayer(784, 64, degree=3)
81     self.ln1 = nn.LayerNorm(64)
82
83     self.kan2 = ChebyKANLayer(64, 32, degree=3)
84     self.ln2 = nn.LayerNorm(32)
85
86     self.kan3 = ChebyKANLayer(32, 10, degree=3)
87
88     def forward(self, x):
89         x = x.view(x.size(0), -1)
90         x = self.kan1(x)
91         x = self.ln1(x)
92         x = self.kan2(x)
93         x = self.ln2(x)
94         x = self.kan3(x)
95         return x
96
97 model_classical_2 = KANModelV2().to(device)
98
99 criterion = nn.CrossEntropyLoss()
100 optimizer = optim.AdamW(model_classical_2.parameters(), lr=1e-3)
101
102 total_params_2 = sum(p.numel() for p in model_classical_2.parameters() if p.requires_grad)
103 print(f"Total trainable parameters in KAN v2: {total_params_2:,}")
104
105 epochs = 15
106 best_test_acc_2 = 0.0
107
108 res2 = {
109     "train_loss": [], "train_acc": [],
110     "test_acc": [], "test_loss": []
111 }
112
113 print("\n--- Training KAN v2 (Chebyshev KAN) ---")
114 for epoch in range(epochs):
115     train_loss, train_acc = train_epoch(model_classical_2, train_loader)
116     test_loss, test_acc = evaluate(model_classical_2, test_loader)
117
118     res2["train_loss"].append(train_loss)
119     res2["train_acc"].append(train_acc)
120     res2["test_loss"].append(test_loss)
121     res2["test_acc"].append(test_acc)
122
123     print(f"Epoch {epoch+1:02d} | Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | Test Acc: {test_acc:.2f}%")
124
125     if test_acc > best_test_acc_2:
126         best_test_acc_2 = test_acc
127         torch.save(model_classical_2.state_dict(), 'best_model_classical_2.pth')
128
129 params2 = count_parameters(model_classical_2)
130 macs2 = compute_macs(model_classical_2)
131 inf2 = measure_inference_time(model_classical_2)
132 acc2 = res2["test_acc"][-1]
133
134 new_row = pd.DataFrame({
135     "Metric": [
136         "Parameters (M)",
137         "MACs (M)",
138         "Inference Time (ms)",
139         "Final Test Accuracy (%)"
140     ],
141     "KAN v2 (Chebyshev)": [
142         params2 / 1e6,
143         macs2 / 1e6,
144         inf2 * 1000,
145         acc2
146     ]
147 })
148
149 comparison_df = comparison_df.merge(new_row, on="Metric")
150 print("\n--- Final Comparison ---")
151 display(comparison_df)
152

```

Total trainable parameters in KAN v2: 263,018

--- Training KAN v2 (Chebyshev KAN) ---

Epoch 01	Loss: 0.4199	Train Acc: 88.36%	Test Acc: 94.50%
Epoch 02	Loss: 0.1625	Train Acc: 95.17%	Test Acc: 96.23%
Epoch 03	Loss: 0.1161	Train Acc: 96.50%	Test Acc: 96.17%
Epoch 04	Loss: 0.0962	Train Acc: 97.09%	Test Acc: 96.85%
Epoch 05	Loss: 0.0794	Train Acc: 97.56%	Test Acc: 96.77%
Epoch 06	Loss: 0.0679	Train Acc: 97.97%	Test Acc: 97.23%
Epoch 07	Loss: 0.0597	Train Acc: 98.08%	Test Acc: 96.90%
Epoch 08	Loss: 0.0533	Train Acc: 98.32%	Test Acc: 97.08%
Epoch 09	Loss: 0.0494	Train Acc: 98.45%	Test Acc: 97.36%
Epoch 10	Loss: 0.0454	Train Acc: 98.57%	Test Acc: 97.40%
Epoch 11	Loss: 0.0369	Train Acc: 98.79%	Test Acc: 97.08%
Epoch 12	Loss: 0.0377	Train Acc: 98.77%	Test Acc: 97.66%
Epoch 13	Loss: 0.0377	Train Acc: 98.75%	Test Acc: 97.44%
Epoch 14	Loss: 0.0287	Train Acc: 99.10%	Test Acc: 97.50%
Epoch 15	Loss: 0.0299	Train Acc: 99.02%	Test Acc: 97.19%

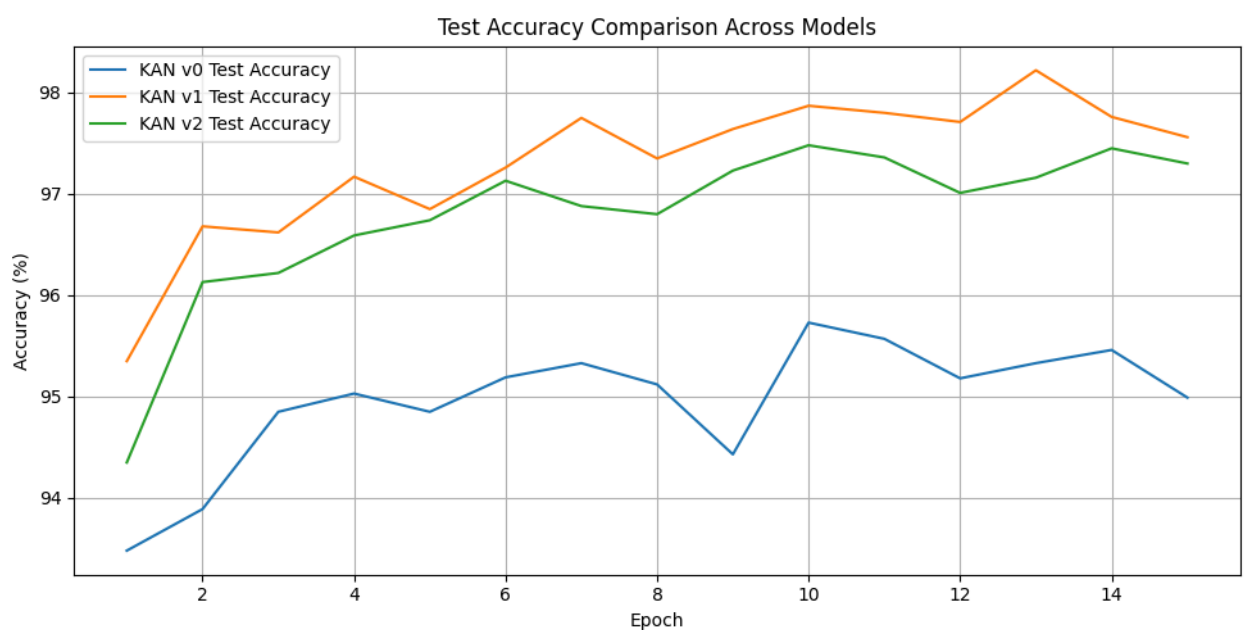
--- Final Comparison ---

	Metric	KAN v0	KAN v1	KAN v2 (Chebyshev)_x	KAN v2 (Chebyshev)_y
0	Parameters (M)	0.418506	1.202014	0.263018	0.263018
1	MACs (M)	0.000704	0.000768	0.000384	0.000384
2	Inference Time (ms)	1.027068	1.518185	2.208345	1.275876
3	Final Test Accuracy (%)	94.990000	97.560000	97.300000	97.190000

✦ model0 vs model1 vs model2

```
1 def plot_comparison(all_results):
2     fig, axes = plt.subplots(2, 1, figsize=(10, 10))
3
4     for model_name, results in all_results.items():
5         epochs = range(1, len(results["test_loss"]) + 1)
6         axes[0].plot(epochs, results["test_loss"], label=f"{model_name} Test Loss")
7     axes[0].set_title("Test Loss Comparison Across Models")
8     axes[0].set_xlabel("Epoch")
9     axes[0].set_ylabel("Loss")
10    axes[0].legend()
11    axes[0].grid(True)
12
13    for model_name, results in all_results.items():
14        epochs = range(1, len(results["test_acc"]) + 1)
15        axes[1].plot(epochs, results["test_acc"], label=f"{model_name} Test Accuracy")
16    axes[1].set_title("Test Accuracy Comparison Across Models")
17    axes[1].set_xlabel("Epoch")
18    axes[1].set_ylabel("Accuracy (%)")
19    axes[1].legend()
20    axes[1].grid(True)
21
22    plt.tight_layout()
23    plt.show()
24
25 all_model_results = {
26     'KAN v0': res0,
27     'KAN v1': res1,
28     'KAN v2': res2
29 }
```

```
1 plot_comparison(all_model_results)
```



Convolutional KAN

convolutional KAN model edge function:

$$y = \text{Conv}(\text{SiLU}(x), W_b) + \text{Conv}([T_0(\tanh x), T_1(\tanh x), \dots, T_D(\tanh x)], W_{\text{poly}})$$

$\text{Conv}(\text{SiLU}(x), W_b)$ = convolution applied to the activated input (base linear component)

$\text{Conv}(\cdot, W_{\text{poly}})$ = convolution applied to the polynomial basis feature maps

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5 import math
6 import matplotlib.pyplot as plt
7
8 from torchvision import datasets, transforms
9 from torch.utils.data import DataLoader
```

```
1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2 print(f"Using device: {device}")
3
4 transform = transforms.Compose([
```

```

5     transforms.ToTensor(),
6     transforms.Normalize((0.1307,), (0.3081,))
7 ])
8
9 train_dataset = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
10 test_dataset = datasets.MNIST(root="./data", train=False, download=True, transform=transform)
11
12 train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)

```

Using device: cuda

```

1 class ChebyKANConv2d(nn.Module):
2     def __init__(self, in_channels, out_channels, kernel_size, stride=1, padding=0, degree=3):
3         super().__init__()
4         self.in_channels = in_channels
5         self.out_channels = out_channels
6         self.kernel_size = kernel_size
7         self.stride = stride
8         self.padding = padding
9         self.degree = degree
10
11         self.base_activation = nn.SiLU()
12
13
14         self.base_weight = nn.Parameter(torch.Tensor(out_channels, in_channels, kernel_size, kernel_size))
15         self.base_bias = nn.Parameter(torch.Tensor(out_channels))
16         self.poly_weight = nn.Parameter(torch.Tensor(out_channels, in_channels * (degree + 1), kernel_size, kernel_size))
17
18         self.reset_parameters()
19
20     def reset_parameters(self):
21         nn.init.kaiming_uniform_(self.base_weight, a=math.sqrt(5))
22         nn.init.zeros_(self.base_bias)
23         nn.init.xavier_normal_(self.poly_weight)
24
25     def forward(self, x):
26         base_out = F.conv2d(self.base_activation(x), self.base_weight, self.base_bias, self.stride, self.padding)
27
28         x_norm = torch.tanh(x)
29
30         basis = [torch.ones_like(x_norm), x_norm]
31         for i in range(2, self.degree + 1):
32             basis.append(2 * x_norm * basis[-1] - basis[-2])
33
34         basis_tensor = torch.cat(basis, dim=1)
35
36         poly_out = F.conv2d(basis_tensor, self.poly_weight, bias=None, stride=self.stride, padding=self.padding)
37
38         return base_out + poly_out

```

```

1 class ChebyKANLayer(nn.Module):
2     def __init__(self, in_features, out_features, degree=3):
3         super().__init__()
4         self.in_features = in_features
5         self.out_features = out_features
6         self.degree = degree
7
8         self.base_activation = nn.SiLU()
9         self.base_weight = nn.Parameter(torch.Tensor(out_features, in_features))
10        self.base_bias = nn.Parameter(torch.Tensor(out_features))
11        self.poly_weight = nn.Parameter(torch.Tensor(out_features, in_features * (degree + 1)))
12
13        self.reset_parameters()
14
15    def reset_parameters(self):
16        nn.init.kaiming_uniform_(self.base_weight, a=math.sqrt(5))
17        nn.init.zeros_(self.base_bias)
18        nn.init.xavier_normal_(self.poly_weight)
19
20    def forward(self, x):

```

```

21     base_out = F.linear(self.base_activation(x), self.base_weight, self.base_bias)
22     x_norm = torch.tanh(x)
23
24     basis = [torch.ones_like(x_norm), x_norm]
25     for i in range(2, self.degree + 1):
26         basis.append(2 * x_norm * basis[-1] - basis[-2])
27
28     basis_tensor = torch.cat(basis, dim=1)
29     poly_out = F.linear(basis_tensor, self.poly_weight)
30     return base_out + poly_out

```

```

1 class ConvChebyKAN(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.conv1 = ChebyKANConv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1, degree=3)
5         self.bn1 = nn.BatchNorm2d(16)
6         self.pool1 = nn.MaxPool2d(2) # 28x28 -> 14x14
7         self.conv2 = ChebyKANConv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1, degree=3)
8         self.bn2 = nn.BatchNorm2d(32)
9         self.pool2 = nn.MaxPool2d(2) # 14x14 -> 7x7
10        self.flatten_dim = 32 * 7 * 7 # 1568
11        self.fc1 = ChebyKANLayer(self.flatten_dim, 64, degree=3)
12        self.ln1 = nn.LayerNorm(64)
13        self.fc2 = ChebyKANLayer(64, 10, degree=3)
14
15    def forward(self, x):
16        x = self.conv1(x)
17        x = self.bn1(x)
18        x = self.pool1(x)
19        x = self.conv2(x)
20        x = self.bn2(x)
21        x = self.pool2(x)
22        x = x.view(x.size(0), -1)
23        x = self.fc1(x)
24        x = self.ln1(x)
25        x = self.fc2(x)
26        return x

```

```

1 chebyKANConvModel = ConvChebyKAN().to(device)
2 criterion = nn.CrossEntropyLoss()
3 optimizer = optim.AdamW(chebyKANConvModel.parameters(), lr=1e-3, weight_decay=1e-4)

```

```

1 def train_epoch(model, loader):
2     model.train()
3     total_loss, correct, total = 0, 0, 0
4
5     for images, labels in loader:
6         images, labels = images.to(device), labels.to(device)
7
8         optimizer.zero_grad()
9         outputs = model(images)
10        loss = criterion(outputs, labels)
11        loss.backward()
12        optimizer.step()
13
14        total_loss += loss.item()
15        _, predicted = outputs.max(1)
16        total += labels.size(0)
17        correct += predicted.eq(labels).sum().item()
18
19    return total_loss / len(loader), 100. * correct / total
20
21 def evaluate(model, loader):
22     model.eval()
23     total_loss, correct, total = 0, 0, 0
24
25     with torch.no_grad():
26         for images, labels in loader:
27             images, labels = images.to(device), labels.to(device)
28             outputs = model(images)
29             loss = criterion(outputs, labels)
30

```

```

31     total_loss += loss.item()
32     _, predicted = outputs.max(1)
33     total += labels.size(0)
34     correct += predicted.eq(labels).sum().item()
35
36     return total_loss / len(loader), 100. * correct / total

```

```

1 epochs = 15
2 history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}
3
4 print(f"Total parameters: {sum(p.numel() for p in chebyKANConvModel.parameters() if p.requires_grad):,}")
5 print("Starting Training...")
6
7 for epoch in range(epochs):
8     train_loss, train_acc = train_epoch(chebyKANConvModel, train_loader)
9     test_loss, test_acc = evaluate(chebyKANConvModel, test_loader)
10
11     history["train_loss"].append(train_loss)
12     history["train_acc"].append(train_acc)
13     history["test_loss"].append(test_loss)
14     history["test_acc"].append(test_acc)
15
16     print(f"Epoch {epoch+1:02d}/{epochs} | "
17           f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.2f}% | "
18           f"Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.2f}%")

```

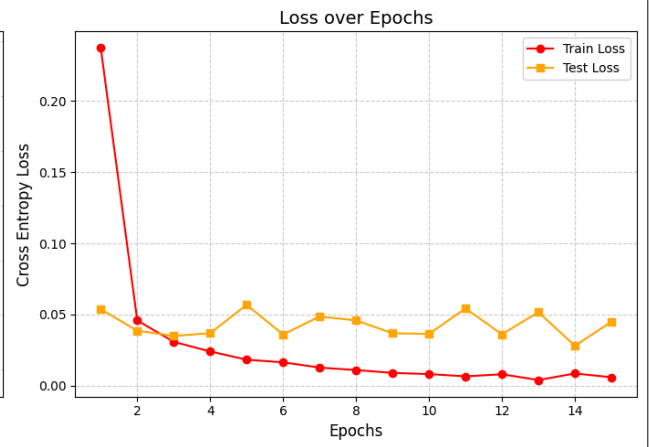
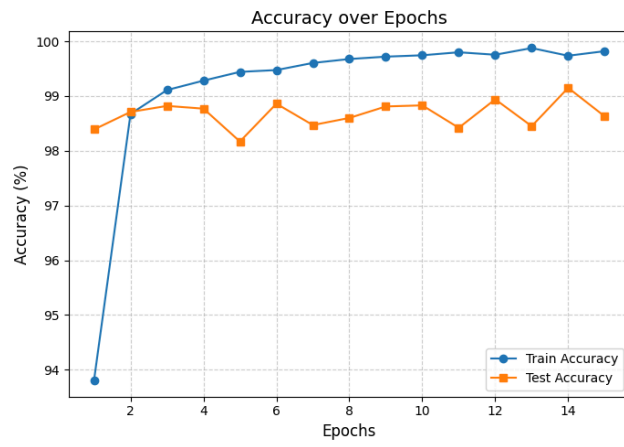
Total parameters: 529,066
Starting Training...

Epoch 01/15	Train Loss: 0.2372	Train Acc: 93.81%	Test Loss: 0.0538	Test Acc: 98.39%
Epoch 02/15	Train Loss: 0.0459	Train Acc: 98.67%	Test Loss: 0.0387	Test Acc: 98.71%
Epoch 03/15	Train Loss: 0.0308	Train Acc: 99.11%	Test Loss: 0.0350	Test Acc: 98.82%
Epoch 04/15	Train Loss: 0.0241	Train Acc: 99.28%	Test Loss: 0.0370	Test Acc: 98.77%
Epoch 05/15	Train Loss: 0.0183	Train Acc: 99.44%	Test Loss: 0.0568	Test Acc: 98.17%
Epoch 06/15	Train Loss: 0.0165	Train Acc: 99.47%	Test Loss: 0.0360	Test Acc: 98.86%
Epoch 07/15	Train Loss: 0.0128	Train Acc: 99.61%	Test Loss: 0.0487	Test Acc: 98.47%
Epoch 08/15	Train Loss: 0.0110	Train Acc: 99.68%	Test Loss: 0.0459	Test Acc: 98.60%
Epoch 09/15	Train Loss: 0.0091	Train Acc: 99.72%	Test Loss: 0.0369	Test Acc: 98.81%
Epoch 10/15	Train Loss: 0.0082	Train Acc: 99.75%	Test Loss: 0.0364	Test Acc: 98.83%
Epoch 11/15	Train Loss: 0.0066	Train Acc: 99.80%	Test Loss: 0.0543	Test Acc: 98.42%
Epoch 12/15	Train Loss: 0.0081	Train Acc: 99.75%	Test Loss: 0.0362	Test Acc: 98.94%
Epoch 13/15	Train Loss: 0.0040	Train Acc: 99.88%	Test Loss: 0.0517	Test Acc: 98.45%
Epoch 14/15	Train Loss: 0.0086	Train Acc: 99.74%	Test Loss: 0.0281	Test Acc: 99.15%
Epoch 15/15	Train Loss: 0.0060	Train Acc: 99.82%	Test Loss: 0.0449	Test Acc: 98.63%

```

1 def plot_learning_curves(history):
2     epochs_range = range(1, len(history["train_loss"]) + 1)
3
4     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
5
6     axes[0].plot(epochs_range, history["train_acc"], label="Train Accuracy", marker='o')
7     axes[0].plot(epochs_range, history["test_acc"], label="Test Accuracy", marker='s')
8     axes[0].set_title("Accuracy over Epochs", fontsize=14)
9     axes[0].set_xlabel("Epochs", fontsize=12)
10    axes[0].set_ylabel("Accuracy (%)", fontsize=12)
11    axes[0].legend()
12    axes[0].grid(True, linestyle='--', alpha=0.6)
13
14    axes[1].plot(epochs_range, history["train_loss"], label="Train Loss", marker='o', color='red')
15    axes[1].plot(epochs_range, history["test_loss"], label="Test Loss", marker='s', color='orange')
16    axes[1].set_title("Loss over Epochs", fontsize=14)
17    axes[1].set_xlabel("Epochs", fontsize=12)
18    axes[1].set_ylabel("Cross Entropy Loss", fontsize=12)
19    axes[1].legend()
20    axes[1].grid(True, linestyle='--', alpha=0.6)
21
22    plt.tight_layout()
23    plt.show()
24
25 plot_learning_curves(history)

```



```

1 def count_parameters(model):
2     return sum(p.numel() for p in model.parameters() if p.requires_grad)
3
4 def compute_macs(model, input_size=(1,1,28,28)):
5     dummy = torch.randn(input_size).to(device)
6     # Compute MACs by passing a dummy input
7     # This is a placeholder for the actual MAC computation logic

```